

1. List

- **Definition:** Ordered, mutable (changeable), allows duplicate values, supports indexing and slicing.
- **Use case:** When you need to maintain order and change elements frequently.

Example and Methods

append() → Adds single element at end

```
a = [12, 67, 34, 90, 5]
```

```
a.append(33)
```

```
print(a) # [12, 67, 34, 90, 5, 33]
```

extend() → Adds elements from another iterable

```
a = [12, 67, 34, 90, 5]
```

```
b = [7, 8, 9]
```

```
a.extend(b)
```

```
print(a) # [12, 67, 34, 90, 5, 7, 8, 9]
```

insert(index, value) → Inserts at given index

```
a.insert(2, 77)
```

```
print(a) # [12, 67, 77, 34, 90, 5, 7, 8, 9]
```

pop([index]) → Removes element at index (default last)

```
a.pop(1)
```

```
print(a) # [12, 77, 34, 90, 5, 7, 8, 9]
```

remove(value) → Removes first occurrence of value

```
a.remove(90)
```

```
print(a)
```

clear() → Empties list

```
a.clear()
```

```
print(a) # []
```

```
# sort() → Sorts in ascending order
```

```
nums = [12, 67, 34, 90, 5, 8, 1]
```

```
nums.sort()
```

```
print(nums) # [1, 5, 8, 12, 34, 67, 90]
```

```
# reverse() → Reverses order
```

```
nums.reverse()
```

```
print(nums)
```

```
# copy() → Creates shallow copy
```

```
b = nums.copy()
```

```
# count(value) → Number of occurrences
```

```
print(nums.count(12))
```

```
# index(value) → First index of value
```

```
print(nums.index(34))
```

```
# max(), min()
```

```
print(max(nums), min(nums))
```

```
# replace via indexing
```

```
nums[0] = 99
```

```
print(nums)
```

2. Dictionary

- **Definition:** Unordered, mutable collection of key-value pairs. Keys must be unique and immutable, values can be anything.
- **Use case:** When data needs to be retrieved via a unique key.

Example and Methods

```
a = {1: "abc", 2: "xyz"}
```

```
# clear()
```

```
a.clear()
```

```
print(a) # {}
```

```
# pop(key) → Removes specific key
```

```
a = {1: "xyz", 2: "abc"}
```

```
a.pop(1)
```

```
print(a) # {2: 'abc'}
```

```
# popitem() → Removes last inserted key-value
```

```
a = {1: "xyz", 2: "abc", 3: "ankit"}
```

```
a.popitem()
```

```
print(a)
```

```
# update()
```

```
a = {1: "abc", 2: "xyz"}
```

```
a.update({3: "jatin"})
```

```
print(a)
```

```
# copy()
```

```
b = a.copy()
```

```
# get(key) → Returns value (no error if not found)
```

```
print(a.get(1))
```

```
# items() → Returns list-like view of (key, value) pairs
```

```
print(a.items())
```

```
# keys(), values()

print(a.keys())

print(a.values())


# fromkeys(iterable, value)

k = ("a", "b")

c = dict.fromkeys(k, 1)

print(c) # {'a': 1, 'b': 1}
```

3. Set

- **Definition:** Unordered, mutable, unique elements only, no duplicates allowed.
- **Use case:** When you need uniqueness and set operations (union, intersection).

Example and Methods

```
s = {1, 2, 3}
```

```
# add()
```

```
s.add(4)
```

```
# update() → Add multiple
```

```
s.update([5, 6])
```

```
# remove(value) → Error if not found
```

```
s.remove(2)
```

```
# discard(value) → No error if not found
```

```
s.discard(10)
```

```
# pop() → Removes random element
```

```
s.pop()
```

```
# clear() → Empty set
```

```
s.clear()
```

```
# set operations
```

```
a, b = {1, 2, 3}, {3, 4, 5}
```

```
print(a.union(b))    # {1, 2, 3, 4, 5}
```

```
print(a.intersection(b)) # {3}
```

```
print(a.difference(b)) # {1, 2}
```

4. String

- **Definition:** Immutable, ordered sequence of characters.
- **Use case:** For storing and manipulating text.

Example and Methods

```
a = "hello world"
```

```
# capitalize() → First letter uppercase
```

```
print(a.capitalize())
```

```
# title()
```

```
print(a.title())
```

```
# center(width)
```

```
print(a.center(20))
```

```
# upper(), lower()
```

```
print(a.upper())
```

```
print(a.lower())
```

```
# join() → Concatenate with separator
```

```
print(" ".join(["hello", "world"]))
```

```
# count(substring)
```

```
print(a.count("l"))
```

```
# index(substring)
```

```
print(a.index("w"))
```

```
# replace(old, new)
```

```
print(a.replace("l", "a"))
```

```
# len()
```

```
print(len(a))
```

```
# split()
```

```
b = a.split()
```

```
print(b)    # ['hello', 'world']
```

```
print(type(b)) # <class 'list'>
```

5. Tuple

- **Definition:** Ordered, immutable, allows duplicates, supports indexing and slicing.
- **Use case:** Fixed data that should not change.

Example and Methods

```
# slicing
```

```
a = "animesh"
```

```
print(a[::-1]) # reverse string using slicing
```

```
# count()
```

```
t = (36, 78, 22, 11, 3, 4, 7, 7, 7, 7)
```

```
print(t.count(7)) # 4
```

```
# index()
```

```
t = (36, 78, 22, 11, 3, 4, 7)
```

```
print(t.index(11)) # 3
```

6. Identity Operators

- Used to check whether two variables point to the same object in memory.

a = 10

b = 10

print(a is b) # True (same memory reference for small ints)

print(a is not b) # False

Final Summary Table

Data Structure	Ordered	Mutable	Allows Duplicates	Indexed	Key Features
List	Yes	Yes	Yes	Yes	Best for ordered, changeable collections
Tuple	Yes	No	Yes	Yes	Immutable sequences
Set	No	Yes	No	No	Unique, unordered elements, set operations
Dictionary	No	Yes	Keys → No, Values → Yes	Keys act as index	Key-value storage
String	Yes	No	Yes	Yes	Immutable text data
